[10]:
```python
print("1.Celsius to Fahrenheit")
print("2.Fahrenheit to Celsius")
choice=input("Enter the choice:")
if choice == "1":
    celcius=float(input("Enter the temparature in celsius:"))
    f=(celcius*9/5)+32
    print("Temparature in Fahrenheit is:",f)
elif choice == "2":
    fahrenheit=float(input("Enter the temparature in fahrenheit:"))
    c=(fahrenheit-32)*5/9
    print("Temperature in celcius is:",c)
```

```
1.Celsius to Fahrenheit
2.Fahrenheit to Celsius
Enter the choice: 1
Enter the temparature in celsius: 32
Temparature in Fahrenheit is: 89.6
```

[13]:
```
a, b, c=10, 12, 20
if a>b and a>c:
    print("A is the largest")
elif b>a and b>c:
    print("B is the largest")
else:
    print("C is the largets")
```

```
C is the largets
```

[40]:
```python
def fact(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * fact(n - 1)
num = int(input("Enter a number: "))
print(f"The factorial of {number} is {fact(number)}")
```

```
Enter a number:  5
The factorial of 5 is 120
```

[42]:
```python
def count_words(sentence: str) -> int:
    words = sentence.split()
    return len(words)

sentence = input("Enter the sentence: ")
print("Number of words:", count_words(sentence))
```

```
Enter the sentence:  Hello World
Number of words: 2
```

[ ]:

```
[60]: a=[1, 2, 3, 4, 5, 6, 7]
      a[:]
[60]: [1, 2, 3, 4, 5, 6, 7]
[61]: a[1:5]
[61]: [2, 3, 4, 5]
[62]: a[1:-5]
[62]: [2]
[63]: a[-5:1]
[63]: []
[64]: a[-1:5:-1]
[64]: [7]
[65]: a[-1:-5]
[65]: []
```

```
[2]: a={1, 2,3, 4, 5}
     print(a)

     {1, 2, 3, 4, 5}

[3]: a={1, 2, 3, 4, 5, 6}
     b={0, 2, 8, 9, 7, 5}

[4]: a.union(b)

[4]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

[5]: a.intersection(b)

[5]: {2, 5}

[6]: a.add(7)

[7]: print(a)

     {1, 2, 3, 4, 5, 6, 7}

[8]: a.difference(b)

[8]: {1, 3, 4, 6}

[9]: b.difference(a)

[9]: {0, 8, 9}

[10]: a.symmetric_difference(b)

[10]: {0, 1, 3, 4, 6, 8, 9}
```

[*]:
```python
def list_operations():
    my_list = []

    while True:
        print("\nList Operations:")
        print("1. Insert an element")
        print("2. Delete an element")
        print("3. Find an element")
        print("4. Display list")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            element = input("Enter element to insert: ")
            my_list.append(element)
            print(f"Element '{element}' inserted.")

        elif choice == 2:
            element = input("Enter element to delete: ")
            if element in my_list:
                my_list.remove(element)
                print(f"Element '{element}' deleted.")
            else:
                print(f"Element '{element}' not found.")

        elif choice == 3:
            element = input("Enter element to find: ")
            if element in my_list:
                print(f"Element '{element}' found.")
            else:
                print(f"Element '{element}' not found.")

        elif choice == 4:
            print("Current List:", my_list)
```

```
[60]: a=[1, 2, 3, 4, 5, 6, 7]
      a[:]

[60]: [1, 2, 3, 4, 5, 6, 7]

[61]: a[1:5]

[61]: [2, 3, 4, 5]

[62]: a[1:-5]

[62]: [2]

[63]: a[-5:1]

[63]: []

[64]: a[-1:5:-1]

[64]: [7]

[65]: a[-1:-5]

[65]: []

[ ]:
```

```
[35]: print(a)
```

```
[1, 0, 2, 3, 4, 5, 6, 7, 8]
```

```
[36]: a.pop(1)
```

```
[36]: 0
```

```
[37]: print(a)
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

```
[38]: a.insert(8, 9)
```

```
[39]: print(a)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[41]: print(a[1:9:2])
```

```
[2, 4, 6, 8]
```

```
[42]: c=a.copy()
```

```
[43]: print(c)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[58]: def list_operations():
          my_list=[Sun, Mon, Tue, Wed, Thu, Fri, Sat]
          while True:
              print("\nList Operations:")
              print("1.Insert an element")
              print("2.Delete an element")
              print("3.Find an element")
              print("4.Display list")
              print("5.Exit")
              choice = int(input("Enter your choice:"))
              if choice == 1:
```

```python
        elif choice == 4:
            print("Current List:", my_list)

        elif choice == 5:
            print("Exiting program...")
            break

        else:
            print("Invalid choice, please try again.")

list_operations()
```

```
List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter your choice:  1
Enter element to insert:  200
Element '200' inserted.

List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter your choice:  2
Enter element to delete:  200
Element '200' deleted.

List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
```

```
[12]: print(a, b)
{1, 2, 3, 4, 5, 6, 7} {0, 2, 5, 7, 8, 9}

[13]: c=a.copy()

[14]: print(a, b, c)
{1, 2, 3, 4, 5, 6, 7} {0, 2, 5, 7, 8, 9} {1, 2, 3, 4, 5, 6, 7}

[24]: a=[1, 2, 3, 4, 5, 6, 7, 8]
      a.pop()

[24]: 8

[25]: print(a)
[1, 2, 3, 4, 5, 6, 7]

[28]: a.append(8)

[29]: print(a)
[1, 2, 3, 4, 5, 6, 7, 8]

[30]: a[0]=0

[33]: print(a)
[0, 2, 3, 4, 5, 6, 7, 8]

[34]: a.insert(0, 1)

[35]: print(a)
[1, 0, 2, 3, 4, 5, 6, 7, 8]
```

```
List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter your choice:  1
Enter element to insert:  20
Element '20' inserted.

List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter your choice:  1
Enter element to insert:  30
Element '30' inserted.

List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter your choice:  4
Current List: ['10, 20, 30, 40, 50', '20', '30']

List Operations:
1. Insert an element
2. Delete an element
3. Find an element
4. Display list
5. Exit
Enter your choice:  ↑↓ for history. Search history with c-↑/c-↓
```

[5]:
```
d={'name': "anu","mark":60,'age':24}
for key,value in d.items():
    print(key,value)
```

```
name anu
mark 60
age 24
```

[6]:
```
d={'name': "anu","mark":60,'age':24}
for key in d:
    print(key,d[key])
```

```
name anu
mark 60
age 24
```

[10]:
```
d.get('name')
```

[10]:
```
'anu'
```

[11]:
```
d.items()
```

[11]:
```
dict_items([('name', 'anu'), ('mark', 60), ('age', 24)])
```

[14]:
```
d.update({'mark':77})
print(d)
```

```
{'name': 'anu', 'mark': 77, 'age': 24}
```

[ ]:

[11]:
```
a1={"py":85, "ds":75}
a2={"os":90}
a1.update(a2)
print(a1)
```

```
{'py': 85, 'ds': 75, 'os': 90}
```

[ ]:

[2]:
```python
import pandas as pd
data = pd.Series([10, 20, 30, 40])
print(data)
```

```
0    10
1    20
2    30
3    40
dtype: int64
```

[8]:
```python
data = pd.Series([10, 20, 30],index=["x","y","z"])
print(data)
```

```
x    10
y    20
z    30
dtype: int64
```

[14]:
```python
import pandas as pd
data={
    "Name": ["Ram", "Vani"],
    "Age": [20,22]
}
df = pd.DataFrame(data)
print(df)
print("\n")
print(df.loc[0])
```

```
   Name  Age
0   Ram   20
1  Vani   22


Name    Ram
Age      20
Name: 0, dtype: object
```

```
Name    Ram
Age      20
Name: 0, dtype: object
```

[17]:
```
import pandas as pd
df=pd.read_csv("Sheets.csv - Sheet1.csv")
print(df)
```

```
      Name  Class  Marks
0   Avishi  Btech   80.0
1    Ragha    BCA   90.0
2    Varun    BCA    NaN
3  Vaibhav    BCA   90.0
```

[18]:
```
df.isnull()
```

[18]:

	Name	Class	Marks
0	False	False	False
1	False	False	False
2	False	False	True
3	False	False	False

[20]:
```
df.dropna()
```

[20]:

	Name	Class	Marks
0	Avishi	Btech	80.0
1	Ragha	BCA	90.0
3	Vaibhav	BCA	90.0

[22]:
```
df.fillna(90)
```

[22]:
```
df.fillna(90)
```

[22]:

	Name	Class	Marks
0	Avishi	Btech	80.0
1	Ragha	BCA	90.0
2	Varun	BCA	90.0
3	Vaibhav	BCA	90.0

[25]:
```
print(df["Marks"].mean())
print(df.Marks.sum())
print(df.Marks.min())
print(df.Marks.max())
print(df.Marks.count())
```

```
86.66666666666667
260.0
80.0
90.0
3
```

[29]:
```
df.shape
```

[29]:
```
(4, 3)
```

[30]:
```
df.ndim
```

[30]:
```
2
```

[31]:
```
len(df)
```

[31]:
```
4
```

[32]:
```
df.head()
```

[32]:

	Name	Class	Marks
0	Avishi	Btech	80.0
1	Ragha	BCA	90.0
2	Varun	BCA	NaN
3	Vaibhav	BCA	90.0

[33]:
```
df.tail()
```

[33]:

	Name	Class	Marks
0	Avishi	Btech	80.0
1	Ragha	BCA	90.0
2	Varun	BCA	NaN
3	Vaibhav	BCA	90.0

[34]:
```
df.index
```

[34]:
```
RangeIndex(start=0, stop=4, step=1)
```

[35]:
```
df.columns
```

[35]:
```
Index(['Name', 'Class', 'Marks'], dtype='str')
```

[36]:
```
import pandas as pd
data={
    "ID":[1, 2, 3, 4],
    "Name":["Ragha", "Manu", "Avishi", "Vaibhav"],
    "Marks":[85, 90, 95, 85]
```

[36]:
```
import pandas as pd
data={
    "ID":[1, 2, 3, 4],
    "Name":["Ragha", "Manu", "Avishi", "Vaibhav"],
    "Marks":[85, 90, 95, 85]
}
df=pd.DataFrame(data)
df.to_csv("students.csv",index=False)
print("CSV file created successfully")
```

```
CSV file created successfully
```

[ ]:

```
[2]: import pandas as pd
data = { 'Name':['Varun','Ragha','Vaibhav'],
         'Age':[25,30,40],
         'Salary':[50000,60000,70000] }
df = pd.DataFrame(data)
print("DataFrame from Scratch:")
print(df)
df['Bonus']=df['Salary']*0.10
print("\nDataFrame after adding 'Bonus' column:")
print(df)
```

```
DataFrame from Scratch:
      Name  Age  Salary
0    Varun   25   50000
1    Ragha   30   60000
2  Vaibhav   40   70000

DataFrame after adding 'Bonus' column:
      Name  Age  Salary   Bonus
0    Varun   25   50000  5000.0
1    Ragha   30   60000  6000.0
2  Vaibhav   40   70000  7000.0
```

[4]:
```python
import pandas as pd
df=pd.read_csv('work experience.csv')
print("First 6 rows of DataFrame")
print(df.head(6))
print("\nColumn names and data types:")
print(df.info())
```

```
First 6 rows of DataFrame
   YearsExperience
0              1.1
1              1.3
2              1.5
3              2.0
4              2.2
5              2.9

Column names and data types:
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 1 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  9 non-null      float64
dtypes: float64(1)
memory usage: 204.0 bytes
None
```

```
[2]: import pandas as pd
data = { 'Name':['Varun','Ragha','Vaibhav'],
         'Age':[25,30,40],
         'Salary':[50000,60000,70000] }
df = pd.DataFrame(data)
print("DataFrame from Scratch:")
print(df)
df['Bonus']=df['Salary']*0.10
print("\nDataFrame after adding 'Bonus' column:")
print(df)
```

```
DataFrame from Scratch:
      Name  Age  Salary
0    Varun   25   50000
1    Ragha   30   60000
2  Vaibhav   40   70000

DataFrame after adding 'Bonus' column:
      Name  Age  Salary   Bonus
0    Varun   25   50000  5000.0
1    Ragha   30   60000  6000.0
2  Vaibhav   40   70000  7000.0
```

[4]:
```python
import pandas as pd
df=pd.read_csv('work experience.csv')
print("First 6 rows of DataFrame")
print(df.head(6))
print("\nColumn names and data types:")
print(df.info())
```

```
First 6 rows of DataFrame
   YearsExperience
0              1.1
1              1.3
2              1.5
3              2.0
4              2.2
5              2.9

Column names and data types:
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 1 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  9 non-null      float64
dtypes: float64(1)
memory usage: 204.0 bytes
None
```

[13]:
```python
import pandas as pd
df=pd.read_csv("values.csv")
print(df)
print("\nMissing values in the DataFrame:")
print(df.isnull().sum())
df.Q1=df['Q1'].fillna(df['Q1'].mean())
print(df.Q1)
```

```
    Year  McDonalds_Revenue_$Billion  Growth_rate_percent   Q1   Q2   Q3   Q4
0   1999                        13.3                  NaN  NaN  NaN  NaN  NaN
1   2000                        14.2                  7.0  NaN  NaN  NaN  NaN
2   2001                        14.9                  4.0  NaN  NaN  NaN  NaN
3   2002                        15.4                  4.0  NaN  NaN  NaN  3.0
4   2003                        17.1                 11.0  3.8  4.3  4.5  4.6
5   2004                        18.6                  8.0  4.4  4.7  4.9  4.5
6   2005                        19.1                  3.0  4.8  5.1  5.3  3.9
7   2006                        20.9                  9.0  4.9  5.4  5.5  5.1
8   2007                        22.8                  9.0  5.3  5.8  5.9  5.8
9   2008                        23.5                  3.0  5.6  6.1  6.3  5.6
10  2009                        22.7                 -3.0  5.1  5.6  6.0  6.0
11  2010                        24.1                  6.0  5.6  5.9  6.3  6.2
12  2011                        27.0                 12.0  6.1  6.9  7.2  6.8
13  2012                        27.6                  2.0  6.5  6.9  7.2  7.0
14  2013                        28.1                  2.0  6.6  7.1  7.3  7.1
15  2014                        27.4                 -2.0  6.7  7.2  7.0  6.6
16  2015                        25.4                 -7.0  6.0  6.5  6.6  6.3
17  2016                        24.6                 -3.0  5.9  6.3  6.4  6.0
18  2017                        22.8                 -7.0  5.7  6.0  5.8  5.3
19  2018                        21.3                 -7.0  5.1  5.4  5.4  5.4
20  2019                        21.4                  1.0  5.0  5.3  5.6  5.4
21  2020                        19.2                -10.0  4.7  3.8  5.4  5.3
22  2021                        23.2                 21.0  5.1  5.9  6.2  6.0
23  2022                        23.2                  0.0  5.7  5.7  5.9  5.9

Missing values in the DataFrame:
Year                          0
McDonalds_Revenue_$Billion    0
Growth_rate_percent           1
```

```
21  2020                        19.2                -10.0  4.7  3.8  5.4  5.3
22  2021                        23.2                 21.0  5.1  5.9  6.2  6.0
23  2022                        23.2                  0.0  5.7  5.7  5.9  5.9

Missing values in the DataFrame:
Year                          0
McDonalds_Revenue_$Billion    0
Growth_rate_percent           1
Q1                            4
Q2                            4
Q3                            4
Q4                            3
dtype: int64
0     5.43
1     5.43
2     5.43
3     5.43
4     3.80
5     4.40
6     4.80
7     4.90
8     5.30
9     5.60
10    5.10
11    5.60
12    6.10
13    6.50
14    6.60
15    6.70
16    6.00
17    5.90
18    5.70
19    5.10
20    5.00
21    4.70
22    5.10
23    5.70
Name: Q1, dtype: float64
```

[18]:
```python
import pandas as pd
df=pd.read_csv("years.csv")
print(df)
print(df.YearsExperience.mean())
print(df.YearsExperience.median())
print(df.YearsExperience.mode())
print(df.YearsExperience.min())
print(df.YearsExperience.max())
print(df.YearsExperience.var())
print(df.YearsExperience.std())
```

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
```

```
10                [illegible]   [illegible]
11                4.0   55794
12                4.0   56957
13                4.1   57081
14                4.5   61111
15                4.9   67938
16                5.1   66029
17                5.3   83088
18                5.9   81363
19                6.0   93940
20                6.8   91738
21                7.1   98273
22                7.9  101302
23                8.2  113812
24                8.7  109431
25                9.0  105582
26                9.5  116969
27                9.6  112635
28               10.3  122391
29               10.5  121872
5.3133333333333335
4.7
0    3.2
1    4.0
Name: YearsExperience, dtype: float64
1.1
10.5
8.055609195402299
2.8378881576627184
```

[ ]:

[1]:
```python
import pandas as pd
df=pd.read_csv("work experience.csv")
print(df)
print(df.describe())
print(df.quantile([0.25,0.5,0.75]))
skewness = df['YearsExperience'].skew()
kurtosis = df['YearsExperience'].kurt()
print(df.skew())
print(df.kurt())
```

```
   YearsExperience
0              1.1
1              1.3
2              1.5
3              2.0
4              2.2
5              2.9
6              3.0
7              3.2
8              3.2
       YearsExperience
count         9.000000
mean          2.266667
std           0.839643
min           1.100000
25%           1.500000
50%           2.200000
75%           3.000000
max           3.200000
      YearsExperience
0.25              1.5
0.50              2.2
0.75              3.0
YearsExperience   -0.18643
dtype: float64
YearsExperience   -1.85304
dtype: float64
```

[2]:
```
import pandas as pd
df=pd.read_csv("work experience.csv")
print(df)
```

```
   YearsExperience
0              1.1
1              1.3
2              1.5
3              2.0
4              2.2
5              2.9
6              3.0
7              3.2
8              3.2
```

[4]:
```
descreption = df.describe()
print(descreption)
print(df.describe())
```

```
       YearsExperience
count         9.000000
mean          2.266667
std           0.839643
min           1.100000
25%           1.500000
50%           2.200000
75%           3.000000
max           3.200000
       YearsExperience
count         9.000000
mean          2.266667
std           0.839643
min           1.100000
25%           1.500000
50%           2.200000
75%           3.000000
max           3.200000
```

[5]:
```
quantiles = df['YearsExperience'].quantile([0.25, 0.5, 0.75])
print(quantiles)
```

```
0.25    1.5
0.50    2.2
0.75    3.0
Name: YearsExperience, dtype: float64
```

[6]:
```
import pandas as pd
data = [10, 20, 20, 30, 100]
print(pd.Series(data).skew())
```

```
2.0286991020803327
```

[7]:
```
skewness = df['YearsExperience'].skew()
print(skewness)
kurtosis = df['YearsExperience'].kurt()
print(kurtosis)
```

```
-0.18643041769017651
-1.8530398729583881
```

[9]:
```
value_counts = df['YearsExperience'].value_counts()
print(value_counts)
df.YearsExperience.value_counts()
```

```
YearsExperience
3.2    2
1.1    1
1.3    1
1.5    1
2.0    1
2.2    1
2.9    1
3.0    1
Name: count, dtype: int64
```

[9]:
```
value_counts = df['YearsExperience'].value_counts()
print(value_counts)
df.YearsExperience.value_counts()
```

```
YearsExperience
3.2    2
1.1    1
1.3    1
1.5    1
2.0    1
2.2    1
2.9    1
3.0    1
Name: count, dtype: int64
```

[9]:
```
YearsExperience
3.2    2
1.1    1
1.3    1
1.5    1
2.0    1
2.2    1
2.9    1
3.0    1
Name: count, dtype: int64
```

[ ]:

[12]:
```python
import pandas as pd
df = pd.read_csv("lap.csv")
print(df)
pd.get_dummies(df)
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder()
enc_data = enc.fit_transform(df[['Category']])
print(enc_data)
print(enc.categories_)
enc_data.toarray()
enc_df = pd.DataFrame(enc_data.toarray())
print(enc_df)
enc_df = pd.DataFrame(emc_data.toarray(), columns = ['2 in 1 Convertible', 'Gaming', 'Netbook', 'Notebook', 'Ultrabook', 'Workstation'])
print(enc_df)
df1 = df.join(enc_df)
print(df1)
```

```
    Manufacturer       Model_Name   Category  Screen_size_inches  \
0          Apple      MacBook Pro  Ultrabook                13.3
1          Apple      Macbook Air  Ultrabook                13.3
2             HP          250 G6   Notebook                15.6
3          Apple      MacBook Pro  Ultrabook                15.4
4          Apple      MacBook Pro  Ultrabook                13.3
..           ...              ...        ...                 ...
972         Dell     Alienware 17     Gaming                17.3
973      Toshiba  Tecra A40-C-1DF   Notebook                14.0
974         Asus        Rog Strix     Gaming                17.3
975           HP      Probook 450   Notebook                15.6
976       Lenovo    ThinkPad T460   Notebook                14.0

                              Screen                         CPU   RAM  \
0    IPS Panel Retina Display 2560x1600        Intel Core i5 2.3GHz   8GB
1                             1440x900        Intel Core i5 1.8GHz   8GB
2                     Full HD 1920x1080   Intel Core i5 7200U 2.5GHz   8GB
3    IPS Panel Retina Display 2880x1800        Intel Core i7 2.7GHz  16GB
```

[13]:
```python
from sklearn.preprocessing import LabelEncoder
```

[13]:
```python
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['RAM'] = le.fit_transform(df['RAM'])
print(df)
print(le.classes_)
```

```
    Manufacturer       Model_Name   Category  Screen_size_inches  \
0          Apple      MacBook Pro  Ultrabook                13.3
1          Apple      Macbook Air  Ultrabook                13.3
2             HP           250 G6   Notebook                15.6
3          Apple      MacBook Pro  Ultrabook                15.4
4          Apple      MacBook Pro  Ultrabook                13.3
..           ...              ...        ...                 ...
972         Dell     Alienware 17     Gaming                17.3
973      Toshiba  Tecra A40-C-1DF   Notebook                14.0
974         Asus        Rog Strix     Gaming                17.3
975           HP      Probook 450   Notebook                15.6
976       Lenovo    ThinkPad T460   Notebook                14.0

                              Screen                         CPU  RAM  \
0    IPS Panel Retina Display 2560x1600        Intel Core i5 2.3GHz    7
1                             1440x900        Intel Core i5 1.8GHz    7
2                      Full HD 1920x1080   Intel Core i5 7200U 2.5GHz    7
3    IPS Panel Retina Display 2880x1800        Intel Core i7 2.7GHz    1
4    IPS Panel Retina Display 2560x1600        Intel Core i5 3.1GHz    7
..                                  ...                         ...  ...
972                    Full HD 1920x1080  Intel Core i7 6700HQ 2.6GHz    4
973                    Full HD 1920x1080   Intel Core i5 6200U 2.3GHz    7
974                    Full HD 1920x1080  Intel Core i7 7700HQ 2.8GHz    1
975          IPS Panel Full HD 1920x1080  Intel Core i5 7200U 2.70GHz    7
976                             1366x768   Intel Core i5 6200U 2.3GHz    5

                  Storage                          GPU Operating_System  \
0               128GB SSD  Intel Iris Plus Graphics 640            macOS
1     128GB Flash Storage        Intel HD Graphics 6000            macOS
2               256GB SSD         Intel HD Graphics 620            No OS
3               512GB SSD            AMD Radeon Pro 455            macOS
4               256GB SSD  Intel Iris Plus Graphics 650            macOS
```

```
972                  Full HD 1920x1080  Intel Core i7 6700HQ 2.6GHz    4
973                  Full HD 1920x1080   Intel Core i5 6200U 2.3GHz    7
974                  Full HD 1920x1080  Intel Core i7 7700HQ 2.8GHz    1
975        IPS Panel Full HD 1920x1080  Intel Core i5 7200U 2.70GHz    7
976                          1366x768   Intel Core i5 6200U 2.3GHz    5

                 Storage                          GPU Operating_System  \
0              128GB SSD  Intel Iris Plus Graphics 640            macOS
1     128GB Flash Storage        Intel HD Graphics 6000            macOS
2              256GB SSD         Intel HD Graphics 620            No OS
3              512GB SSD            AMD Radeon Pro 455            macOS
4              256GB SSD  Intel Iris Plus Graphics 650            macOS
..                   ...                            ...              ...
972  256GB SSD +  1TB HDD       Nvidia GeForce GTX 1070          Windows
973            256GB SSD         Intel HD Graphics 520          Windows
974  256GB SSD +  1TB HDD       Nvidia GeForce GTX 1060          Windows
975  128GB SSD +  1TB HDD          Nvidia GeForce 930MX          Windows
976          508GB Hybrid         Intel HD Graphics 520          Windows

    Operating_System_Version  Weight_kg        Price
0                        NaN       1.37  11912523.48
1                        NaN       1.34   7993374.48
2                        NaN       1.86   5112900.00
3                        NaN       1.83  22563005.40
4                        NaN       1.37  16037611.20
..                       ...        ...          ...
972                       10       4.42  24897600.00
973                       10       1.95  10492560.00
974                       10       2.73  18227710.80
975                       10       2.04   8705268.00
976                        7       1.70   8909784.00

[977 rows x 13 columns]
['12GB' '16GB' '24GB' '2GB' '32GB' '4GB' '6GB' '8GB']
```

[ ]:

[14]:
```
import pandas as pd
data=[1,2,3]
df = pd.DataFrame(data)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
print(scaled_data)
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)
print(scaled_data)
from sklearn.preprocessing import Normalizer
scaler = Normalizer()
normalized_data = scaler.fit_transform(df)
print(normalized_data)
```

```
[[-1.22474487]
 [ 0.        ]
 [ 1.22474487]]
[[0. ]
 [0.5]
 [1. ]]
[[1.]
 [1.]
 [1.]]
```

[15]:
```
import pandas as pd
data=pd.read_csv("years.csv")
df = pd.DataFrame(data)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
print(scaled_data)
print (" ")
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
```

[15]:
```python
import pandas as pd
data=pd.read_csv("years.csv")
df = pd.DataFrame(data)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
print(scaled_data)
print (" ")
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)
print(scaled_data)
print (" ")
from sklearn.preprocessing import Normalizer
scaler = Normalizer()
normalized_data = scaler.fit_transform(df)
print(normalized_data)
```

```
[[-1.51005294 -1.36011263]
 [-1.43837321 -1.10552744]
 [-1.36669348 -1.419919  ]
 [-1.18749416 -1.20495739]
 [-1.11581443 -1.33978143]
 [-0.86493538 -0.71830716]
 [-0.82909552 -0.58815781]
 [-0.75741579 -0.79981746]
 [-0.75741579 -0.42881019]
 [-0.57821647 -0.69801306]
 [-0.50653674 -0.47433279]
 [-0.47069688 -0.74976858]
 [-0.47069688 -0.70662043]
 [-0.43485702 -0.70201994]
 [-0.29149756 -0.55250402]
 [-0.1481381  -0.29921736]
 [-0.07645838 -0.37004264]
 [-0.00477865  0.26285865]
```

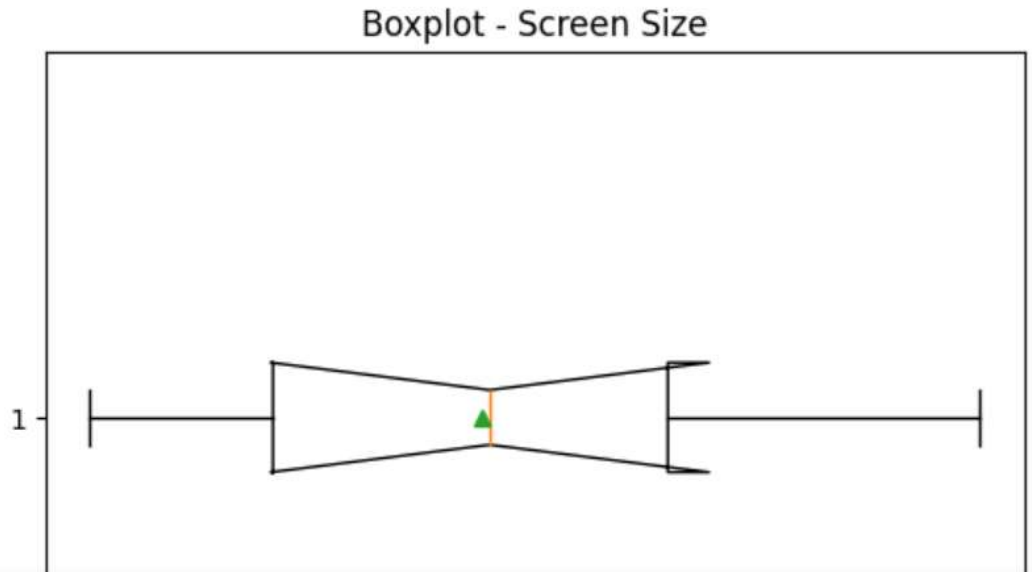
```
[[2.79592303e-05 1.00000000e+00]
 [2.81354832e-05 1.00000000e+00]
 [3.97551085e-05 9.99999999e-01]
 [4.59506031e-05 9.99999999e-01]
 [5.51502844e-05 9.99999998e-01]
 [5.11987570e-05 9.99999999e-01]
 [4.98753117e-05 9.99999999e-01]
 [5.87749104e-05 9.99999998e-01]
 [4.96547443e-05 9.99999999e-01]
 [6.46977564e-05 9.99999998e-01]
 [6.16912903e-05 9.99999998e-01]
 [7.16922965e-05 9.99999997e-01]
 [7.02284178e-05 9.99999998e-01]
 [7.18277534e-05 9.99999997e-01]
 [7.36364973e-05 9.99999997e-01]
 [7.21245840e-05 9.99999997e-01]
 [7.72387888e-05 9.99999997e-01]
 [6.37877911e-05 9.99999998e-01]
 [7.25145334e-05 9.99999997e-01]
 [6.38705555e-05 9.99999998e-01]
 [7.41241359e-05 9.99999997e-01]
 [7.22477179e-05 9.99999997e-01]
 [7.79846398e-05 9.99999997e-01]
 [7.20486414e-05 9.99999997e-01]
 [7.95021518e-05 9.99999997e-01]
 [8.52418023e-05 9.99999996e-01]
 [8.12181003e-05 9.99999997e-01]
 [8.52310558e-05 9.99999996e-01]
 [8.41565145e-05 9.99999996e-01]
 [8.61559666e-05 9.99999996e-01]]
```

[ ]:

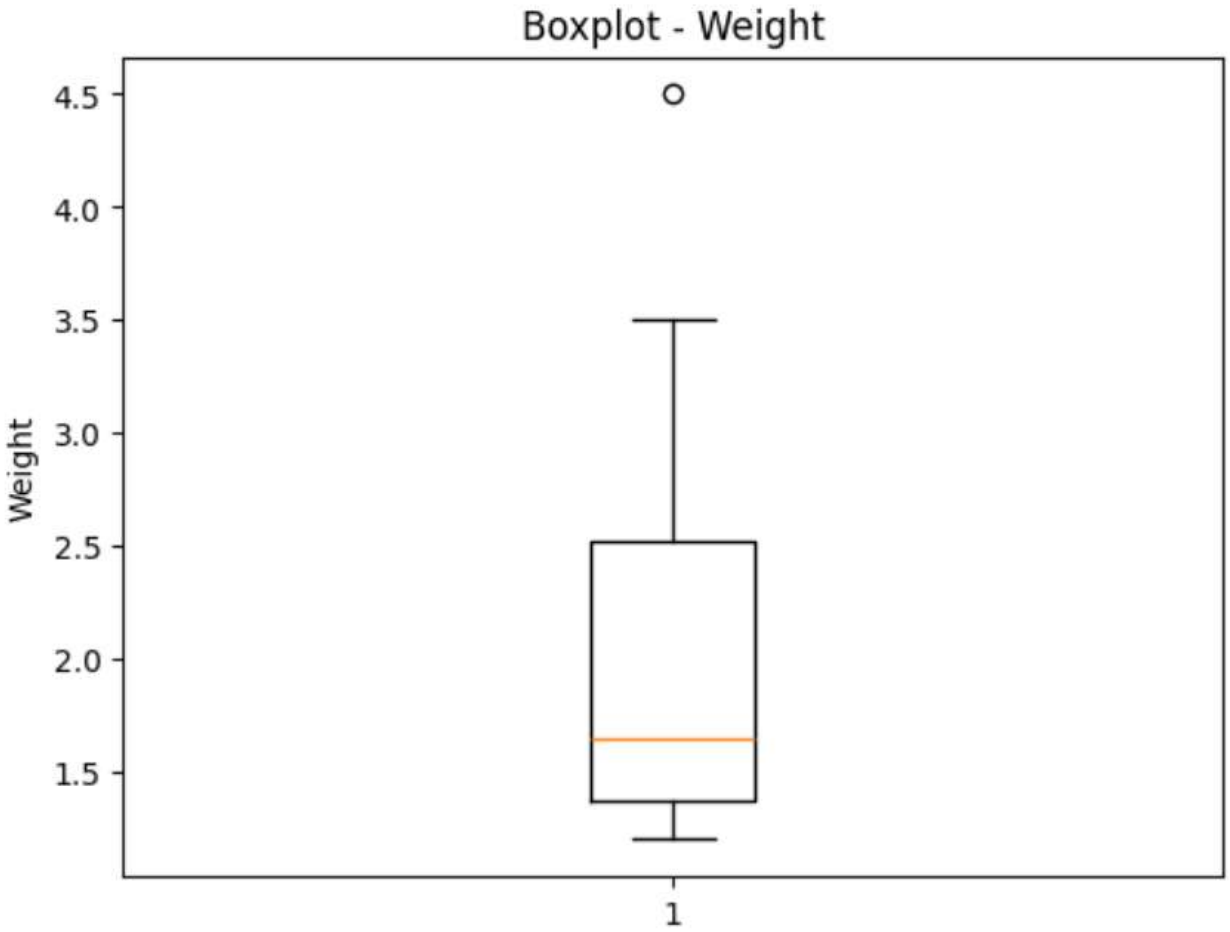
[2]:
```python
import pandas as pd

emp=pd.DataFrame({
    'ID':[101,102,103,104,105],
    'Name':['A','B','C','D','E'],
    'Dept':['HR','Eng','Eng','HR','Mkt']
})

sal=pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Sal': [50000, 70000, 80000, 55000, 60000]
})

sales1=pd.DataFrame({
     'Date': pd.date_range(start='2026-01-01', periods=5),
    'Reg': ['N']*5,
     'Sale': [250, 300, 200, 400, 350]
})

sales2 = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01', periods=5),
    'Reg': ['S']*5,
    'Sale': [300, 320, 280, 360, 310]
})

print("Employee Data:")
print(emp)

print("\nSalary Data:")
print(sal)

merge_data = pd.merge(emp, sal, on='ID')
```

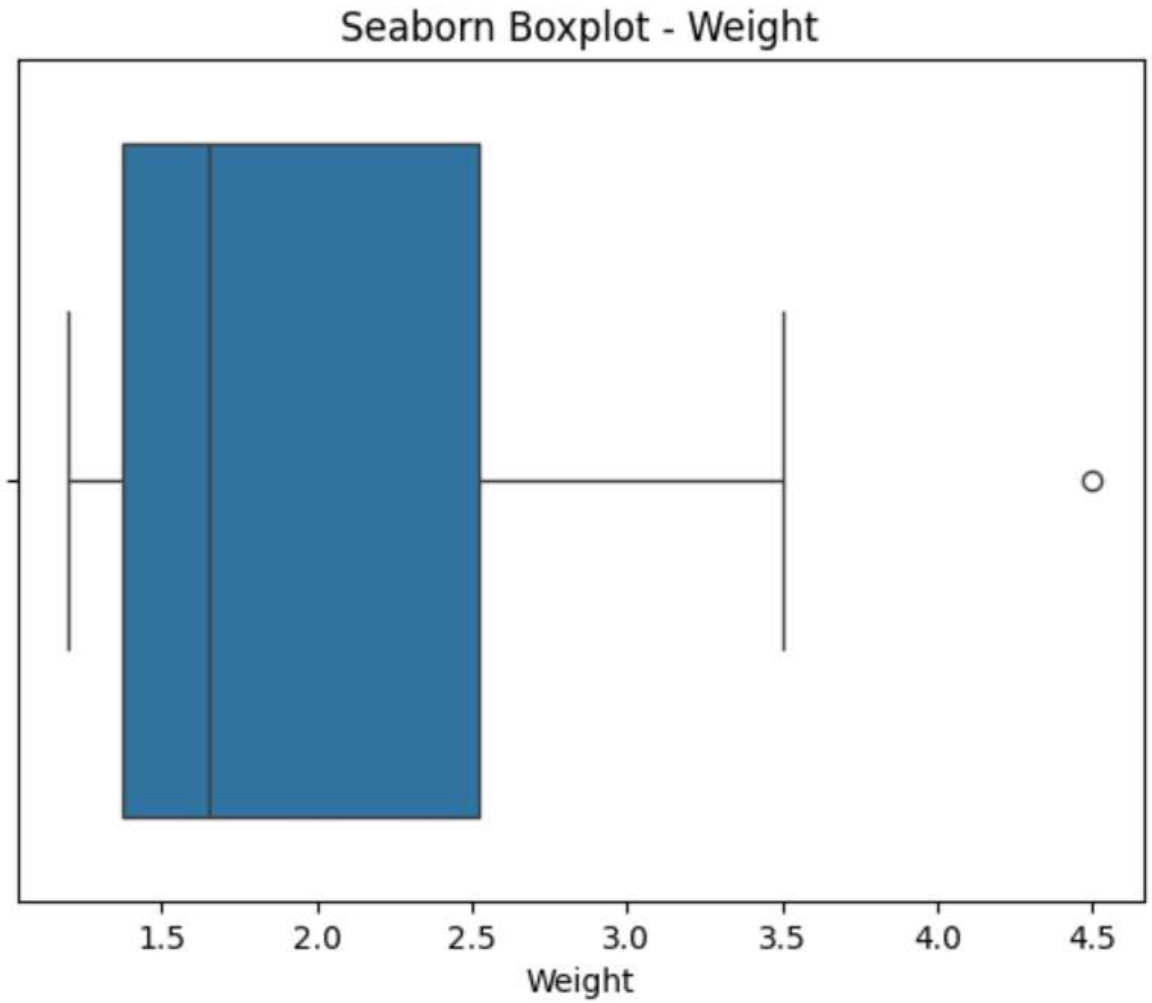
```python
print("\nMerged Data:")
print(merge_data)


avg = emp.merge(sal, on='ID').groupby('Dept')['Sal'].mean() .reset_index()
avg.columns=['dept','avgsal']

print("\nAverage Salary by Dept:")
print(avg)

emp1 = emp.set_index('ID')
sal1 = sal.set_index('ID')
join_data = emp1.join(sal1,how='inner')

print("\nJoined Data:")
print(join_data)

final = pd.concat([sales1, sales2], axis=1)

print("\nConcatenated Sales Data:")
print(final)
```

```
Dataset:
      Name  Screen  Weight  RAM   Price
0     Dell    15.6     1.8    8   55000
1       HP    14.0     1.5    8   50000
2   Lenovo    15.6     2.2   16   65000
3     Asus    13.3     1.3    8   48000
4     Acer    17.0     3.5   16   70000
5    Apple    13.0     1.2    8   90000
6      MSI    15.6     4.5   32  120000
7  Samsung    14.0     1.4    8   52000

Boxplot for Screen Size
```

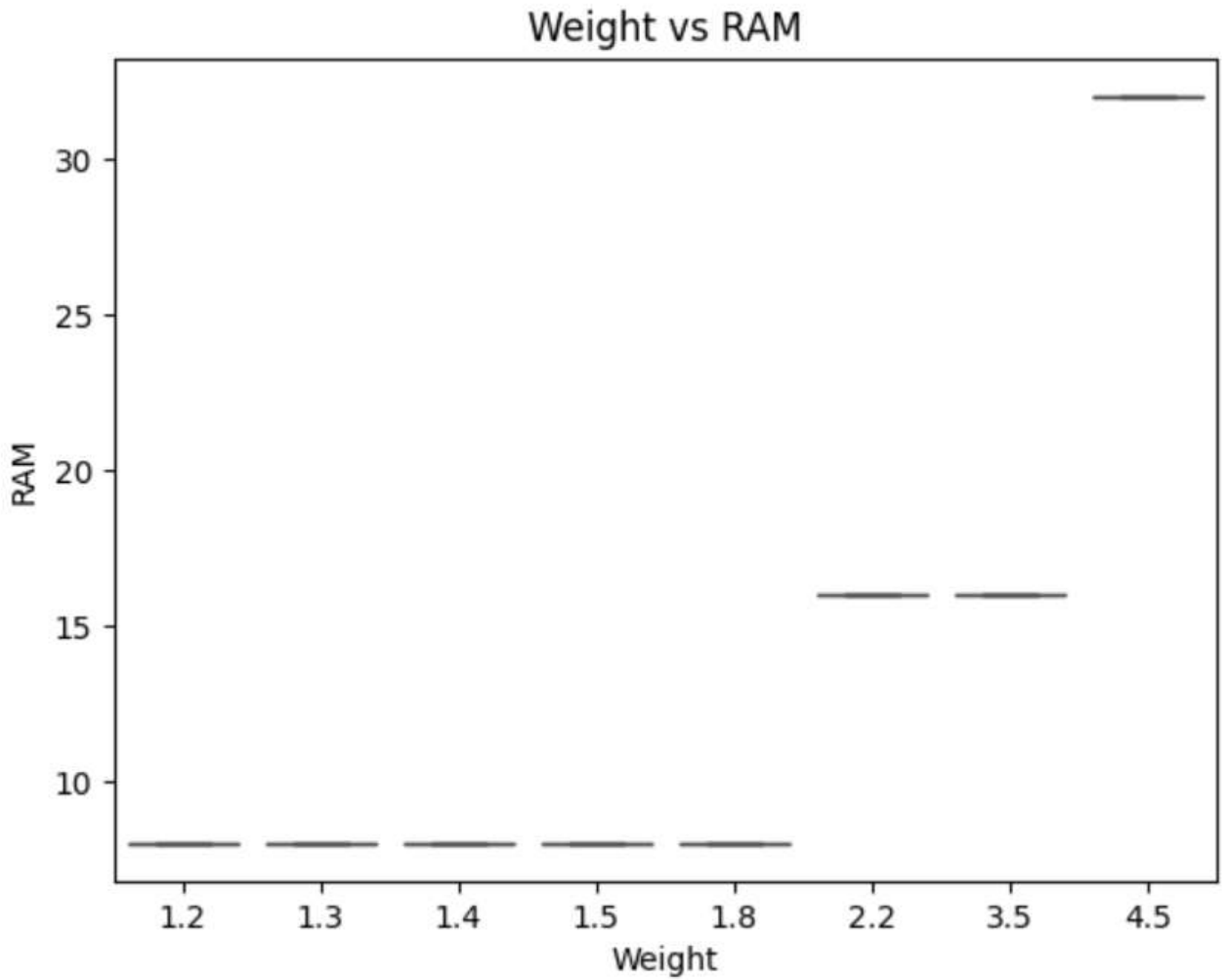


```
Boxplot for Weight
```

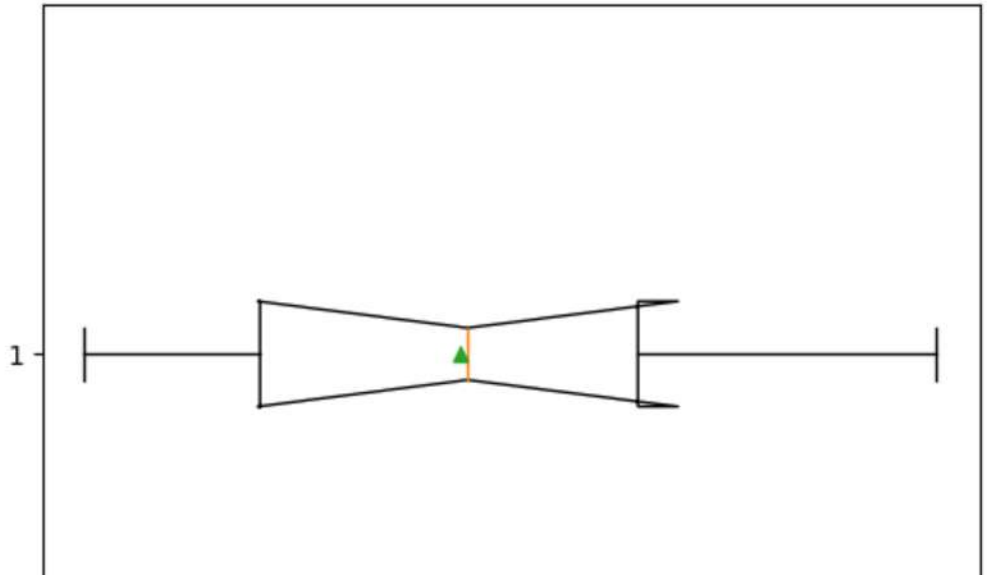


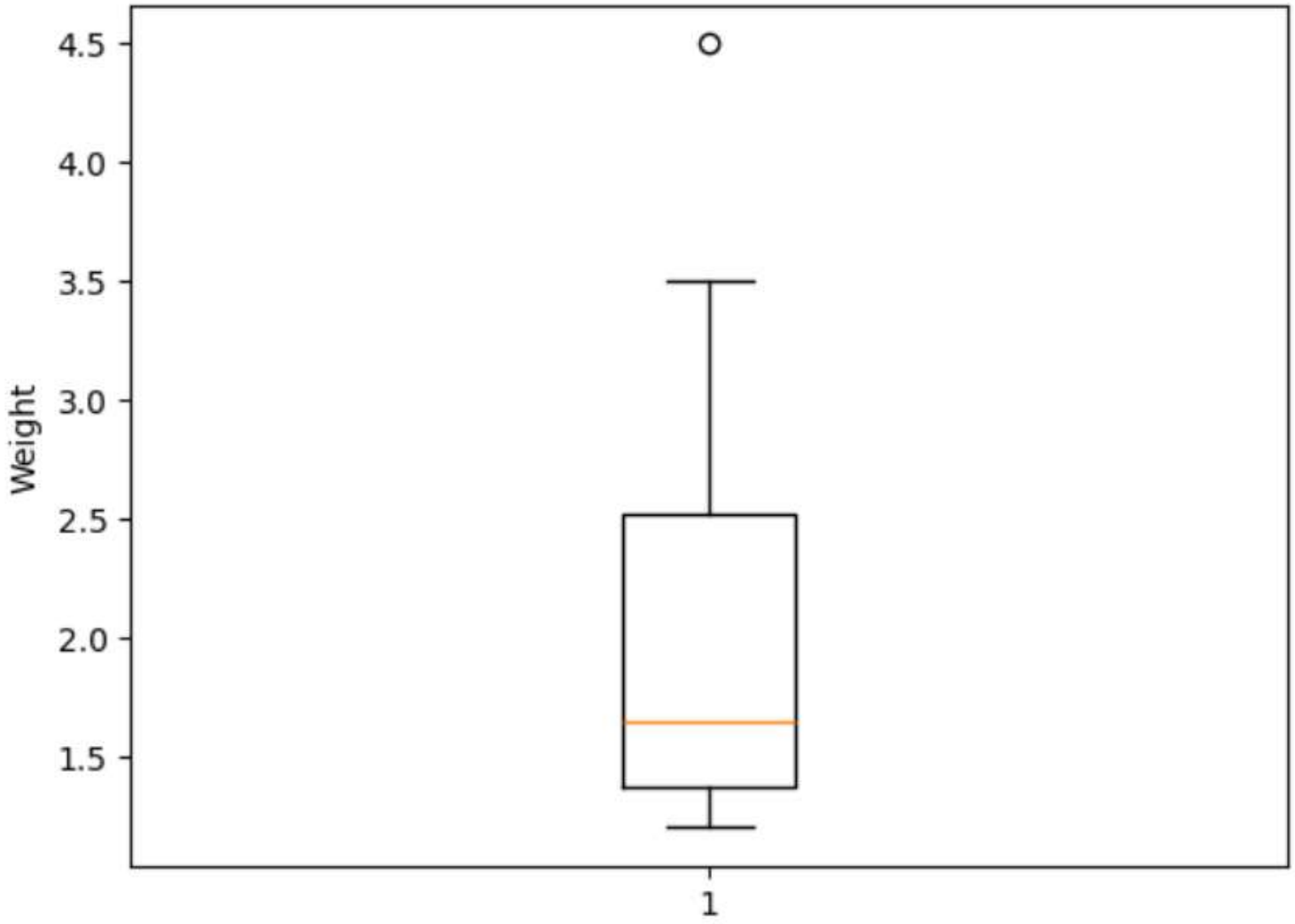

Seaborn Boxplot - Weight
1.5
2.0
2.5
3.0
3.5
4.0
4.5
Weight

```
Seaborn Boxplot - Weight vs RAM
```

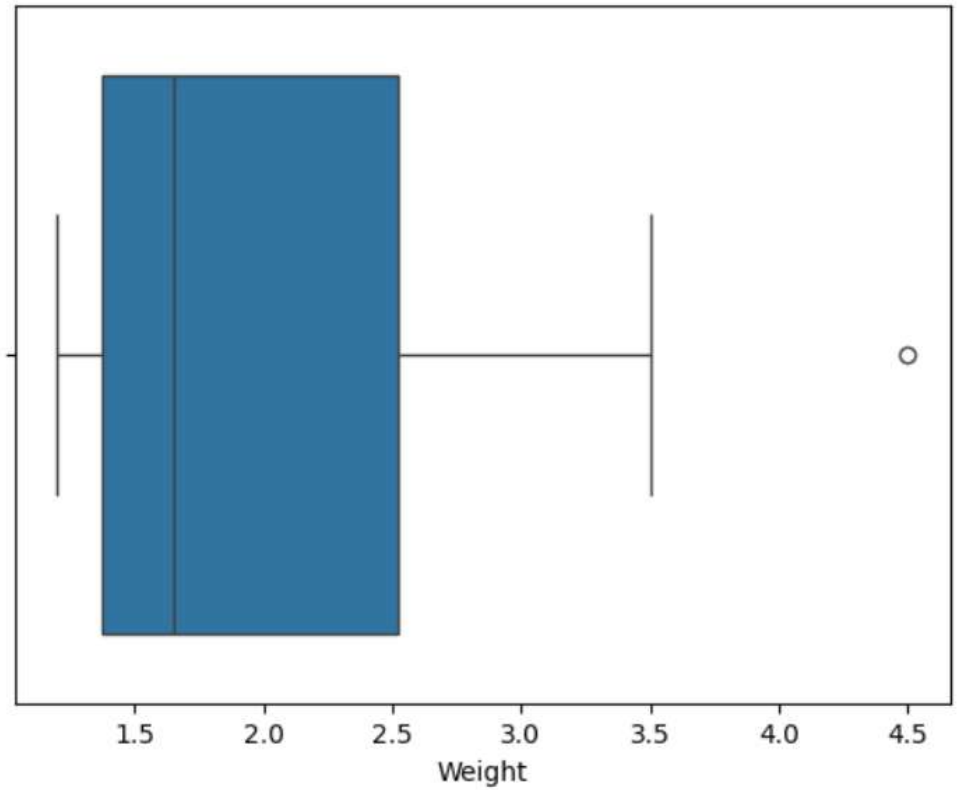


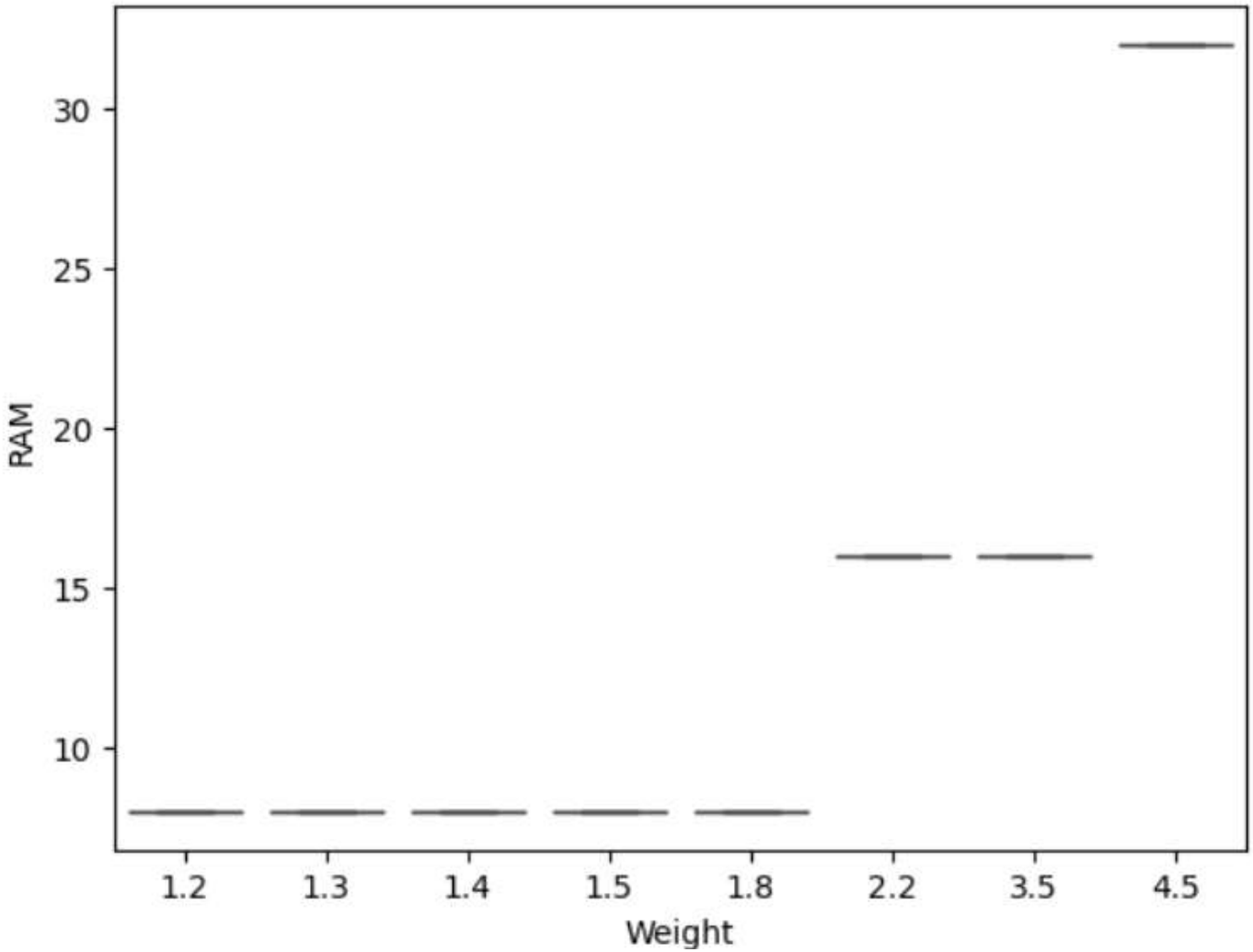
[4]:
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("laptop.csv")

plt.boxplot(df["Screen"], notch=True, vert=False, showmeans=True)  # notch(curve) around the median
plt.xlabel("Screen")
plt.show()

plt.boxplot(df["Weight"])
plt.ylabel("Weight")
plt.show()
```




4.5
4.0
3.5
3.0
2.5
2.0
1.5
Weight
1

[5]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df["Weight"])
plt.show()

sns.boxplot(x=df["Weight"], y=df["RAM"])
plt.show()
```




30
25
20
15
10
RAM
1.2
1.3
1.4
1.5
1.8
2.2
3.5
4.5
Weight

[6]:
```
Q1 = df["Weight"].quantile(0.25)
Q3 = df["Weight"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["Weight"] < lower) | (df["Weight"] > upper)]

print(outliers)
```

```
  Name  Screen  Weight  RAM   Price
6  MSI    15.6     4.5   32  120000
```

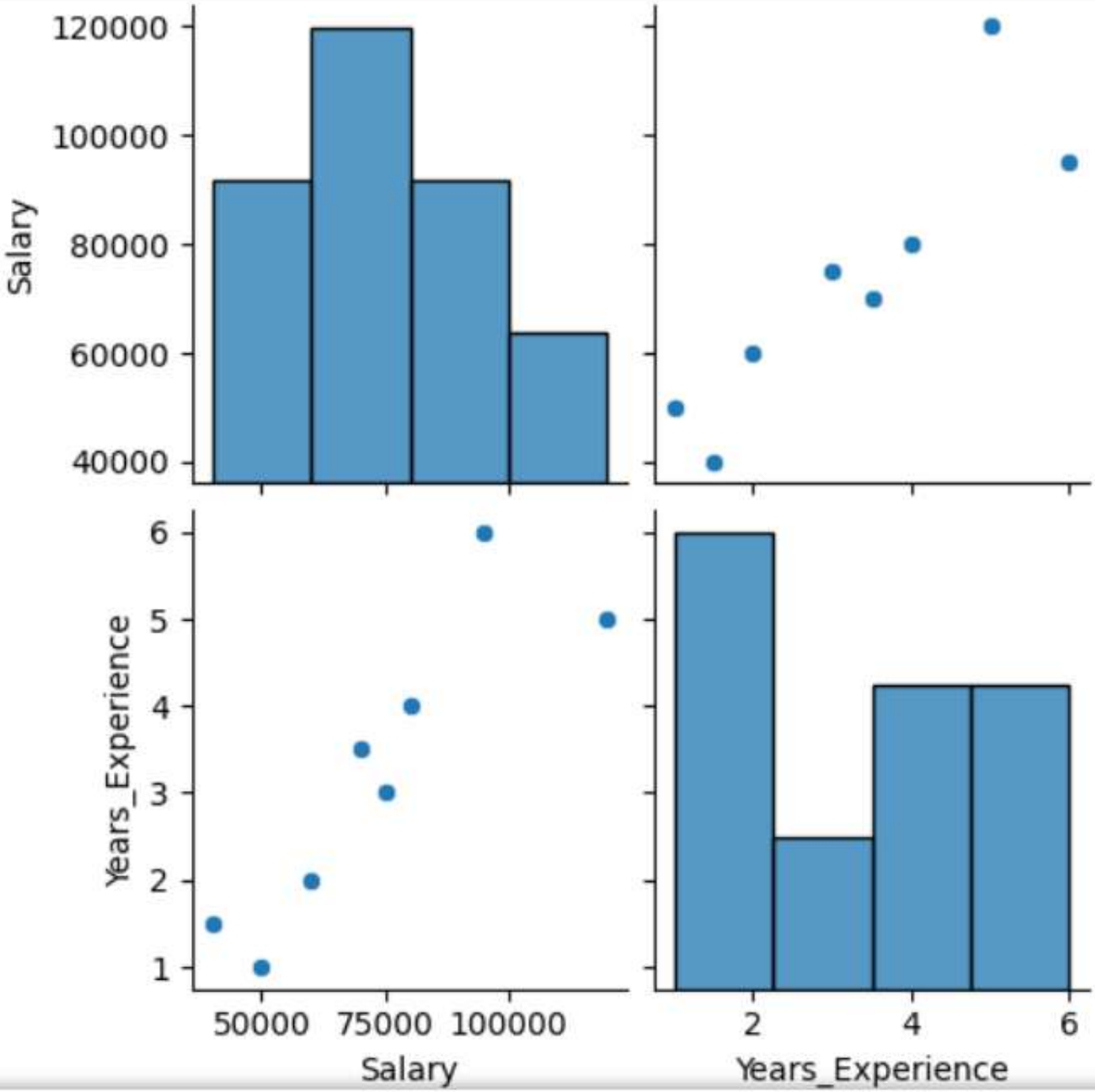
[2]:
```python
import pandas as pd
emp = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Name': ['A', 'B', 'C', 'D', 'E'],
    'Dept': ['HR', 'Eng', 'Eng', 'HR', 'Mkt']
})

sal = pd.DataFrame({
    'ID': [101, 102, 103, 104, 105],
    'Sal': [50000, 70000, 80000, 55000, 60000]
})

sales1 = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01', periods=5),  # this fun, genetrate the date
    'Reg': ['N']*5,  # same as ['N','N','N','N','N']
    'Sale': [250, 300, 200, 400, 350]
})

sales2 = pd.DataFrame({
    'Date': pd.date_range(start='2026-01-01', periods=5),
    'Reg': ['S']*5,
    'Sale': [300, 320, 280, 360, 310]
})

print("Employee Data:")
print(emp)

print("\nSalary Data:")
print(sal)


# Merging


merge_data = pd.merge(emp, sal, on='ID')

print("\nMerged Data:")
print(merge_data)
```

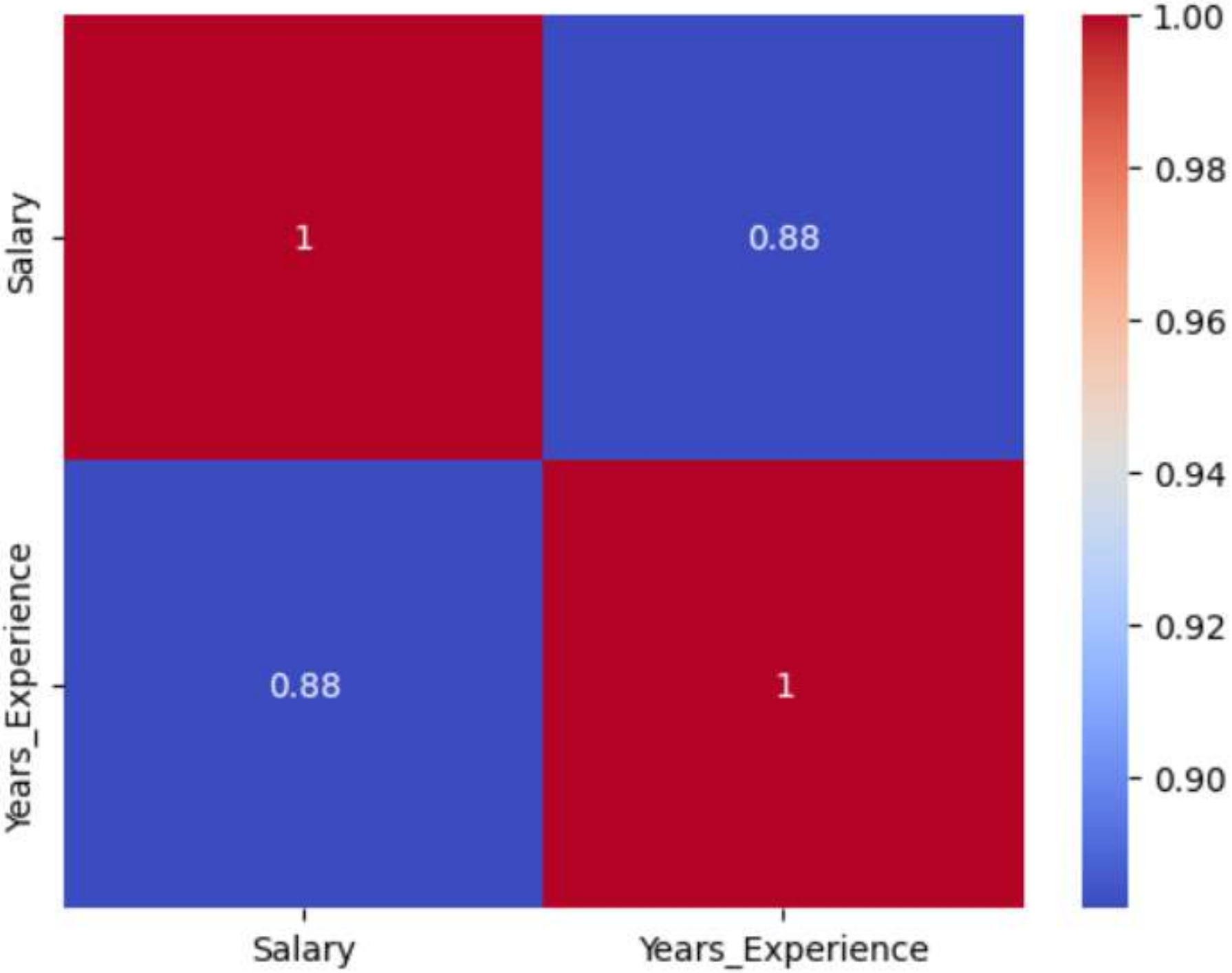
```python
merge_data = pd.merge(emp, sal, on='ID')

print("\nMerged Data:")
print(merge_data)


avg = emp.merge(sal, on='ID').groupby('Dept')['Sal'].mean() .reset_index()
avg.columns=['dept','avgsal']

print("\nAverage Salary by Dept:")
print(avg)
emp1 = emp.set_index('ID')
sal1 = sal.set_index('ID')
join_data = emp1.join(sal1,how='inner')

print("\nJoined Data:")
print(join_data)

final = pd.concat([sales1, sales2], axis=1)

print("\nConcatenated Sales Data:")
print(final)
```

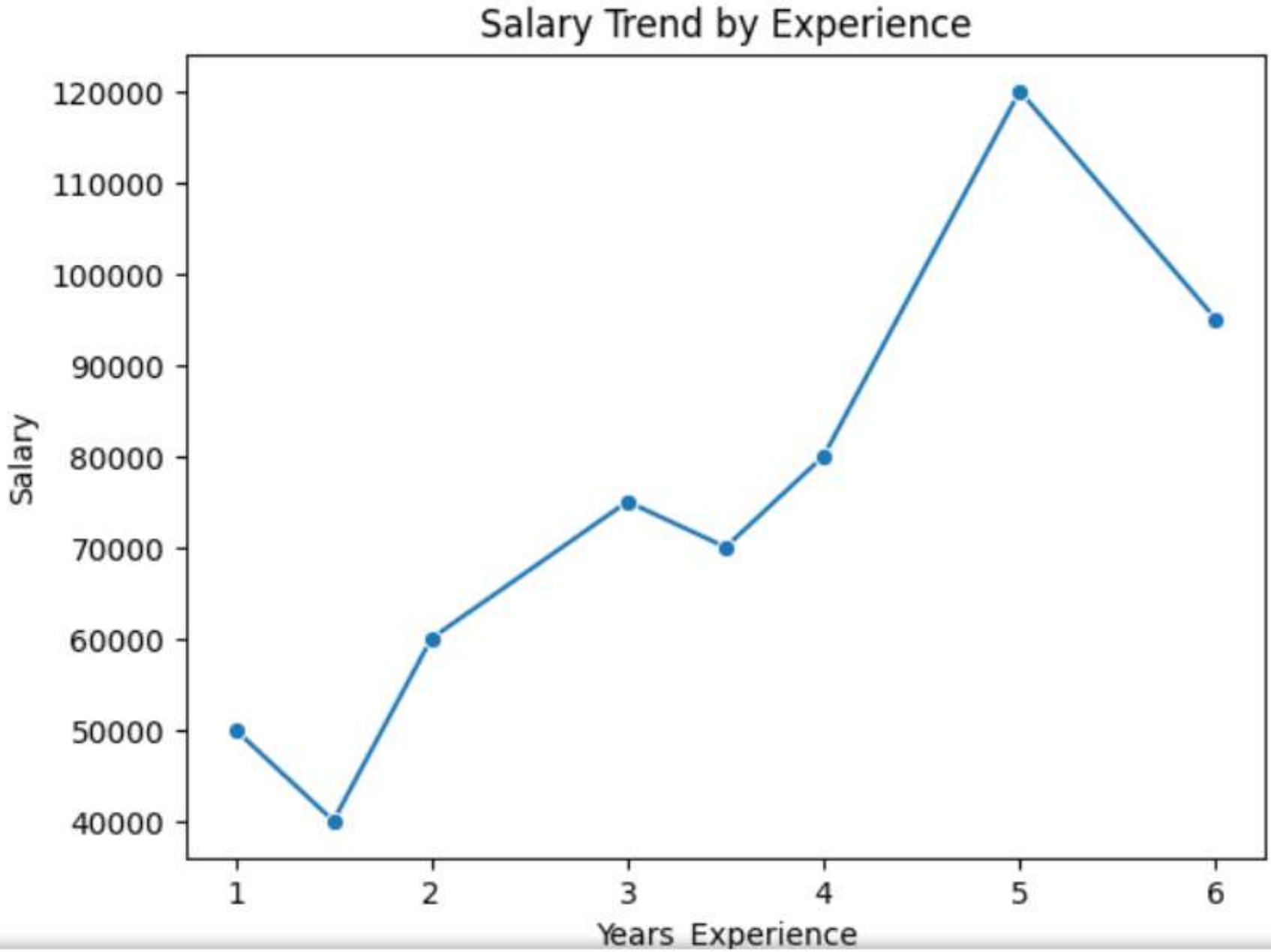
```
Employee Data:
    ID Name Dept
0  101    A   HR
1  102    B  Eng
2  103    C  Eng
3  104    D   HR
4  105    E  Mkt

Salary Data:
    ID    Sal
0  101  50000
1  102  70000
2  103  80000
3  104  55000
4  105  60000

Merged Data:
    ID Name Dept    Sal
```

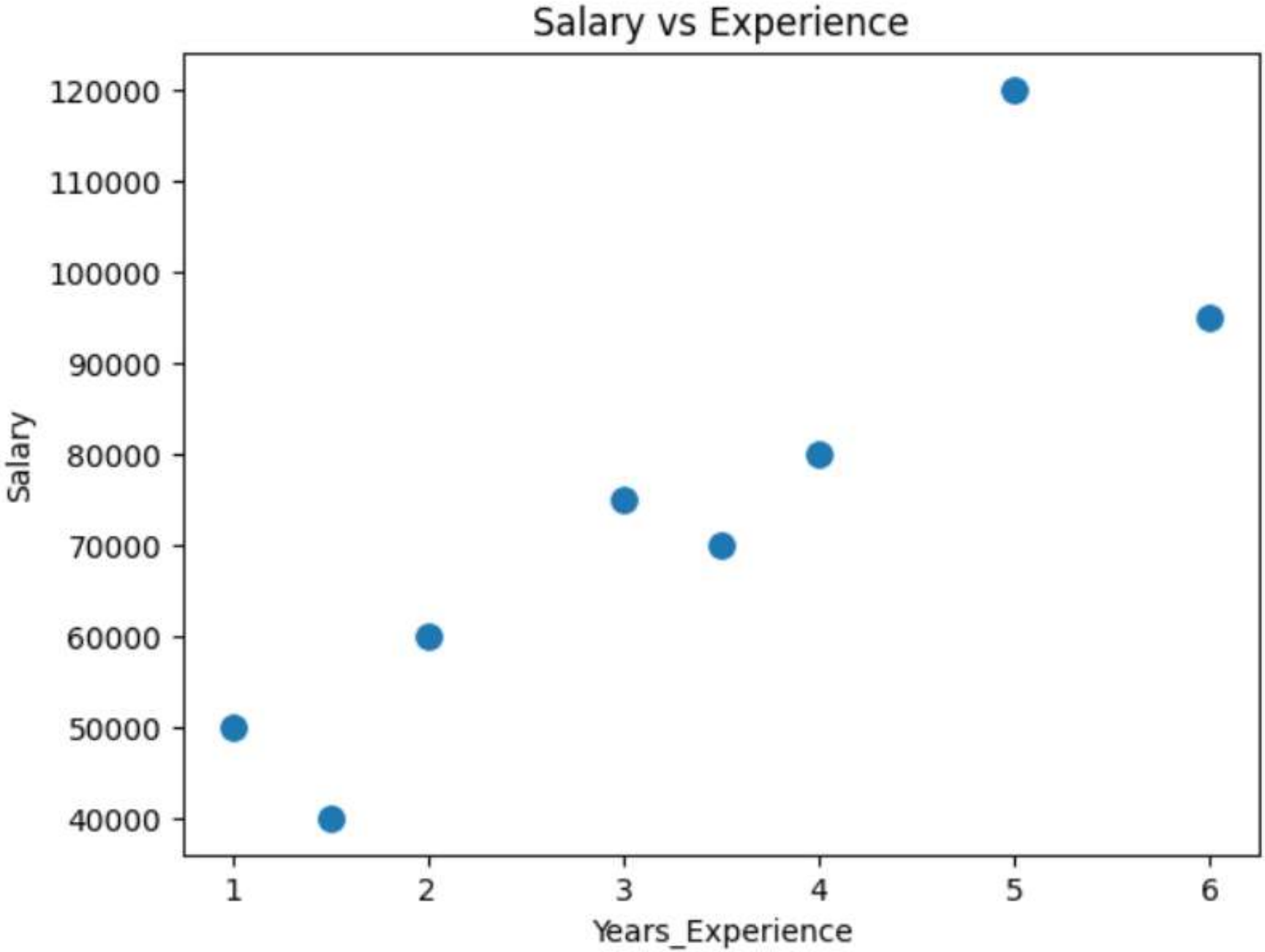
[3]:
```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
'Salary': [50000, 60000, 75000, 80000, 120000, 95000, 40000, 70000],
'Years_Experience': [1, 2, 3, 4, 5, 6, 1.5, 3.5],
'Experience_Level': ['Junior', 'Junior', 'Mid', 'Mid', 'Senior', 'Senior', 'Junior', 'Mid']
}
df = pd.DataFrame(data)

sns.histplot(df['Salary'], kde=True)
plt.title("Salary Distribution")
plt.show()
sns.pairplot(df[['Salary', 'Years_Experience']])
plt.show()
sns.heatmap(df[['Salary', 'Years_Experience']].corr(), annot=True, cmap='coolwarm')
plt.show()

sns.lineplot(x='Years_Experience', y='Salary', data=df, marker='o')

plt.title("Salary Trend by Experience")
plt.show()
sns.scatterplot(x='Years_Experience', y='Salary', data=df, s=100)

plt.title("Salary vs Experience")
plt.show()
sns.barplot(x='Experience_Level', y='Salary', data=df)
plt.title("Salary by Experience Level")
plt.show()
```

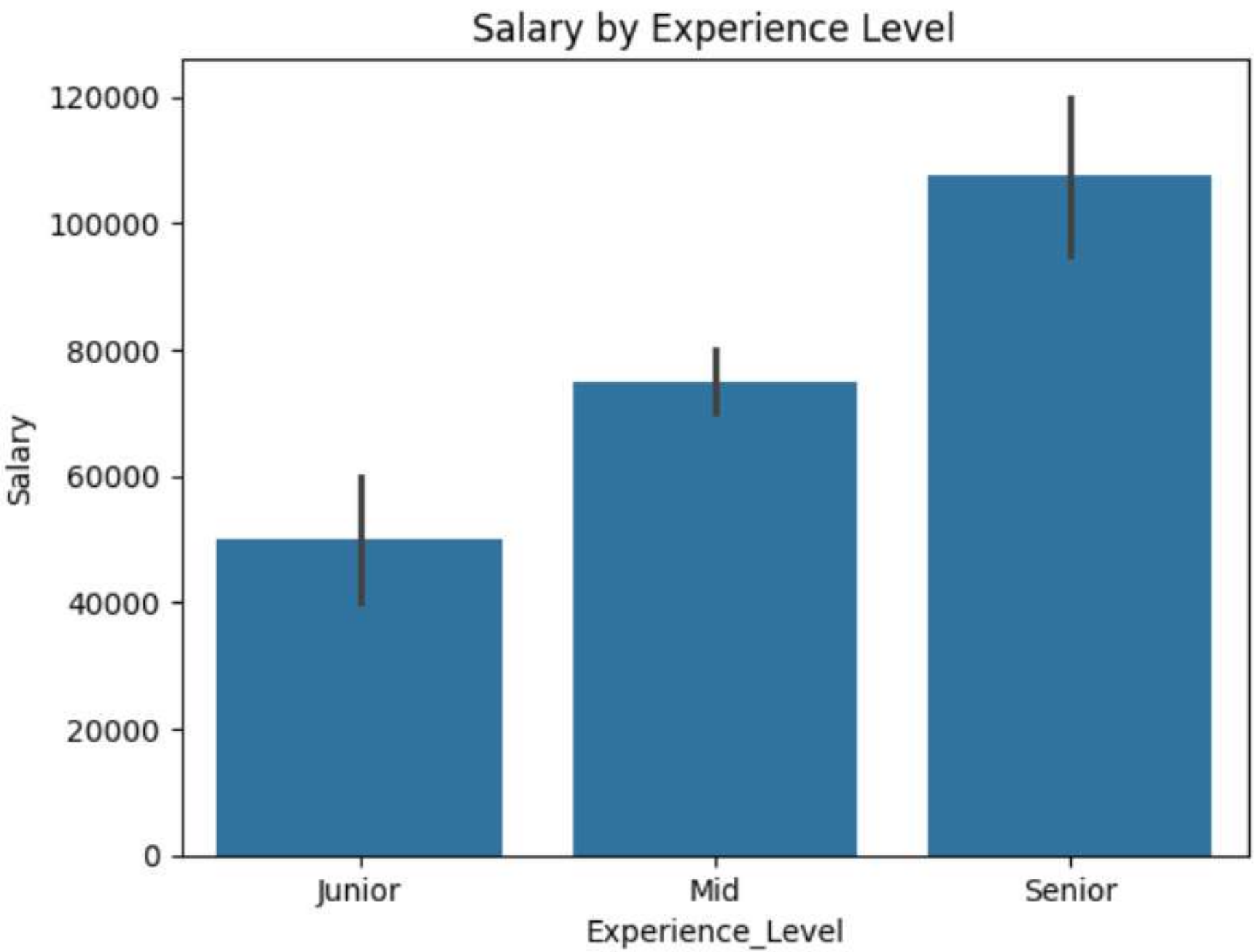
Salary Distribution
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Count
40000
50000
60000
70000
80000
90000
100000
110000
120000
Salary


120000
100000
80000
60000
40000
Salary
6
5
4
3
2
1
Years_Experience
50000
75000
100000
Salary
2
4
6
Years_Experience


Salary
Years_Experience
1
0.88
0.88
1
Salary
Years_Experience
1.00
0.98
0.96
0.94
0.92
0.90


Salary Trend by Experience
120000
110000
100000
90000
80000
70000
60000
50000
40000
Salary
1
2
3
4
5
6
Years Experience


Salary vs Experience
120000
110000
100000
90000
80000
70000
60000
50000
40000
Salary
1
2
3
4
5
6
Years_Experience


Salary by Experience Level
120000
100000
80000
60000
40000
20000
0
Salary
Junior
Mid
Senior
Experience_Level